WEEK 13

Security and Testing

Two lessons, one mindset — for Web Applications Development

What we cover today

- **Two lessons, one mindset**
  - Lesson 1 — Web Application Security Basics
  - Lesson 2 — Web and Mobile Applications Testing
- **Why these two together?**
  - Security is what an attacker tries to break
  - Testing is how we prove, on every change, that nothing is broken
  - Both ask the same question: how could this go wrong?

> **Key Takeaway**
> *Every input is hostile until proven otherwise; every assumption is wrong until a test confirms it.*

LESSON 1

Web Application Security Basics

Confidentiality, integrity, availability — and the mindset that protects them.

What security actually means

- **The CIA triad — three pillars**
  - Confidentiality — only authorised people see the data
  - Integrity — data is not changed without permission
  - Availability — the service stays up under load and attack
- **All three matter — losing any one is a security failure**
  - A leaked database breaks confidentiality
  - An attacker editing prices breaks integrity
  - A denial-of-service attack breaks availability

> **Key Takeaway**
> *If a feature passes a security review on confidentiality alone, it has not really passed.*

The mindset shift

- **Two ways to look at the same line of code**
  - Beginner asks: will this work for a polite user?
  - Engineer asks: will this hold against a hostile user?
- **The difference between a feature and a vulnerability**
  - The polite user types maria@example.es
  - The hostile user types ' OR '1'='1' or <script>
  - Same input field — completely different outcome

> **Key Takeaway**
> *Security is not added at the end. It is a question asked at the start of every line of code.*

OWASP Top 10 — what it is

- **OWASP — Open Worldwide Application Security Project**
  - Non-profit foundation, vendor-neutral
  - Publishes the canonical list of common vulnerabilities
- **The Top 10 list**
  - Updated roughly every four years
  - Current edition: OWASP Top 10 (2021)
  - Free to read, free to use, no licence cost
- **Used by every regulator, auditor and security consultant**

> **Key Takeaway**
> *The Top 10 is not a checklist — it is a map of where bugs hide.*

OWASP Top 10 — categories (1/2)

- **A01 — Broken Access Control**
  - Users see or change data they should not
- **A02 — Cryptographic Failures**
  - Sensitive data stored or sent without proper encryption
- **A03 — Injection**
  - Untrusted input concatenated into a query or command
- **A04 — Insecure Design**
  - The feature is broken before any code is written
- **A05 — Security Misconfiguration**
  - Defaults left on, admin panels accidentally exposed

OWASP Top 10 — categories (2/2)

- **A06 — Vulnerable and Outdated Components**
  - A library has a known CVE and was never updated
- **A07 — Identification and Authentication Failures**
  - Login flow lets attackers in or breaks user identity
- **A08 — Software and Data Integrity Failures**
  - Code or data updated from an untrusted source
- **A09 — Security Logging and Monitoring Failures**
  - Attacks happen and nobody notices for weeks
- **A10 — Server-Side Request Forgery (SSRF)**
  - Server fetches a URL the user supplied

> **Key Takeaway**
>
> *Print the list. Stick it next to your screen. Refer to it on every code review.*

SQL Injection — the vulnerability

- **What it is**
  - User input is concatenated into a database query as a string
  - Attacker types input that ends the query early and starts a new one
- **Where it lives**
  - Login forms, search boxes, contact forms, anywhere user input reaches the database
  - Any route that builds SQL by string concatenation is suspect
- **Why it still happens in 2026**
  - Tutorials and copy-paste code that predate parameterised queries

SQL Injection — the classic payload

- **The payload**
  - ' OR '1'='1
- **Pasted into a username field, the query becomes**
  - SELECT * FROM users WHERE username = '' OR '1'='1' AND ...
  - The OR '1'='1' makes the WHERE clause always true
  - The login is bypassed completely — no password needed
- **A trivial input has just compromised every account**

SQL Injection — the defence

- **Parameterised queries — also called prepared statements**
  - The application sends the query and the values separately
  - The database treats values as data, never as code
- **Every modern database driver supports them**
  - Python sqlite3, psycopg, mysql.connector — all support ? or %s placeholders
  - Node.js pg, Prisma, Sequelize — all support $1 or ? placeholders
- **There is no excuse to concatenate user input into SQL**

> **Key Takeaway**
> *Never put user input directly into a query string. Use the placeholder, every time, without exception.*

Cross-Site Scripting (XSS) — the vulnerability

- **What it is**
  - Application takes user input and renders it into a page without escaping
  - Attacker types JavaScript inside the input
  - Every visitor's browser then runs that JavaScript
- **What the attacker's script can do**
  - Read cookies, hijack sessions, redirect to phishing pages
  - Modify what the page shows
  - Make HTTP requests on behalf of the victim

XSS — three flavours

- **Stored XSS**
  - Payload saved in the database
  - Served to every visitor — worst case
- **Reflected XSS**
  - Payload bounced off a URL parameter
  - Often delivered via a phishing link
- **DOM-based XSS**
  - Payload manipulated entirely in the browser
  - Caused by client-side JavaScript that trusts URL fragments or input fields

XSS — the defence

- **Output encoding — escape at the moment of insertion**
  - Convert < to < ; > to > ; " to "
  - Apply this every time user data lands in HTML
- **Use the framework's safe primitives**
  - Plain JavaScript: textContent, never innerHTML
  - React and Vue: {value} curly-brace interpolation
  - Jinja and Twig: the |escape filter (often default)
- **Add a Content-Security-Policy header as defence in depth**

> **Key Takeaway**
> *Treat HTML output the way you treat SQL queries — escape every user-influenced value at the moment of insertion.*

Password storage — a short history

- **Plaintext (1970s and earlier)**
  - The literal characters. Never use.
- **Unsalted MD5 / SHA-1 (mid-1990s)**
  - Fast one-way hashes — broken by rainbow tables
- **Salted SHA-256 (late 2000s)**
  - Per-user random salt — but GPUs compute billions per second
- **bcrypt / scrypt / argon2 (2000 onwards)**
  - Deliberately slow with a tunable cost factor — current standard

Password storage — current standard

- **bcrypt — the safe default**
  - Built-in salt, tunable cost factor, available everywhere
  - Tune the cost so a single hash takes about 250 ms on your server
- **argon2id — the future-proof choice**
  - Memory-hard, GPU-resistant, designed for modern attackers
  - Recommended by OWASP for new applications
- **What never to do**
  - Never encrypt — encryption is reversible; hashing is not
  - Never log a password, even in error traces

> **Key Takeaway**
> *Passwords are radioactive — handle them with bcrypt, never look at them, dispose of them as soon as hashed.*

Sessions — best practices

- **Generate a fresh session identifier on login**
  - Cryptographically random, at least 128 bits of entropy
  - Store in a cookie, never in the URL
- **Mark the cookie correctly**
  - HttpOnly — JavaScript cannot read it
  - Secure — only sent over HTTPS
  - SameSite=Lax — protects against CSRF
- **Rotate the identifier on login and on privilege change**
- **Expire on logout and after inactivity**

Multi-Factor Authentication (MFA)

- **Three things authentication can prove**
  - Something you know — a password
  - Something you have — a phone, a hardware key
  - Something you are — fingerprint, face
- **MFA combines two of the three**
  - Most common: password plus a code from an authenticator app
  - Stronger: hardware keys following the FIDO2 standard
- **Why it matters**
  - A leaked password alone is no longer enough to compromise an account

> **Key Takeaway**
> *MFA turns a credential leak from a disaster into an inconvenience.*

HTTPS and TLS — three guarantees

- **HTTPS = HTTP wrapped in TLS (Transport Layer Security)**
- **Encryption**
  - Third parties on the network see ciphertext, not your data
- **Integrity**
  - Modified packets are detected and rejected
- **Authentication**
  - The certificate proves the server is who it claims to be
- **All three must hold — losing any one breaks the contract**

HTTPS and TLS — implementation

- **Get a certificate from a trusted Certificate Authority**
  - Let's Encrypt — free, automated, default choice for new sites
  - Renews automatically through Certbot or your hosting provider
- **Force HTTP-to-HTTPS redirects**
  - Return 301 or 308 from every plain-HTTP request
- **Add the HSTS header — Strict-Transport-Security**
  - Tells browsers to refuse plaintext to your site again
  - Specify a long max-age once you trust your TLS setup

Essential security headers

- **Strict-Transport-Security (HSTS)**
  - Force HTTPS for the configured duration
- **Content-Security-Policy (CSP)**
  - Declare which scripts and resources are allowed
  - Strongest defence-in-depth against XSS
- **X-Content-Type-Options: nosniff**
  - Stop browsers from guessing MIME types
- **Referrer-Policy: strict-origin-when-cross-origin**
  - Avoid leaking internal URLs to third parties

> **Key Takeaway**
> *Headers are free to add and stop entire classes of attack — there is no reason not to set them.*

Lesson 1 recap — the five rules

- **Validate every input on the server**
  - Client-side validation is a courtesy, never a defence
- **Parameterise every query**
  - No string concatenation — ever
- **Escape every output**
  - Especially HTML — at the moment of insertion
- **Hash every password with bcrypt**
  - Hash, never encrypt; never store plaintext
- **Wrap everything in HTTPS**
  - Free, automatic — no excuse left in 2026

> **Key Takeaway**
> *Five rules. Print them. Live by them. They block most of the OWASP Top 10 by themselves.*

LESSON 2

Web and Mobile Applications Testing

How professionals stop fearing their own code: automated, layered, in CI.

Why we test

- **The cost-of-bugs principle**
  - Cost roughly multiplies by ten at each stage
  - Development → review → CI → staging → production
- **What testing is not**
  - Not running the app and clicking around
  - That is manual exploration — useful, but not testing
- **What a test actually is**
  - An automated, executable specification
  - Fails loudly when the behaviour drifts

> **Key Takeaway**
> *Tests are not a tax on shipping — they are how you ship faster, because you stop fearing your own changes.*

The cost-of-bugs curve

- **A bug fixed while typing**
  - Costs minutes — caught by the editor's red squiggle
- **A bug fixed in code review**
  - Costs an hour — caught by a colleague
- **A bug fixed in CI before merge**
  - Costs a few hours — caught by the test suite
- **A bug fixed in production**
  - Costs days plus reputation damage
- **Tests pull bugs leftward on this curve**

The Testing Pyramid

- **Three layers, one shape**
  - End-to-End (E2E) — few, slow, expensive — top
  - Integration — some, medium speed, medium cost — middle
  - Unit — many, fast, cheap — bottom
- **More tests at the bottom, fewer at the top**
- **Inverting the pyramid breaks the suite**
  - Lots of E2E tests are slow and flaky
  - Few unit tests miss bugs that should be caught early

Key Takeaway

Build a pyramid, not an ice-cream cone. The shape matters more than the absolute count.

Pyramid layers — what each tests

- **Unit tests**
  - One pure function or class in isolation
  - No database, no network — milliseconds each
  - Hundreds to thousands of them
- **Integration tests**
  - Several units plus one real dependency
  - Usually the database or the framework
  - Tens to low hundreds — fractions of a second each
- **End-to-End tests**
  - The whole system through a real browser or device
  - Roughly ten — seconds to a minute each

AAA — anatomy of a good test

- **Arrange — set up the inputs and the world**
  - Construct objects, seed the database, mock external services
- **Act — call the function under test**
  - Once. Exactly once. Multiple acts hide which one failed.
- **Assert — check the outputs**
  - Use the framework's assertion library — assertEqual, expect, assert
  - Assert on outcomes, not implementation details
- **AAA reads like a tiny three-act play**

> **Key Takeaway**
>
> *A test is a tiny story: given this, when I do that, then this happens.*

FIRST — properties of trustworthy tests

- **Fast**
  - A test taking 10 seconds to fail will not be run
- **Isolated**
  - No dependence on other tests, run order, or external services
- **Repeatable**
  - Same inputs, same result — always
- **Self-validating**
  - Pass or fail — never look at the output and judge
- **Timely**
  - Written before or with the production code, not afterwards

> **Key Takeaway**
> *FIRST is the difference between a test suite engineers trust and one they ignore.*

Test naming

- **A test name is a sentence**
  - test_picks_the_lowest_price reads as documentation
  - test_rejects_zero_prices says exactly what it asserts
- **Anonymous names are a code smell**
  - test1 — what does it test?
  - test_works — works under what conditions?
- **The reader should understand the test's purpose without opening it**

Unit testing in practice

- **File structure**
  - One test file per production module, mirroring the directory tree
  - tests/checkout_test.py for src/checkout.py
- **Tools by language**
  - Python — unittest (built-in) or pytest (community standard)
  - JavaScript — Jest or Vitest
- **Where most of your bugs will actually be caught**
  - But only if you write a lot of them

What unit tests do not catch

- **Wiring mistakes between modules**
  - A route calling the wrong function
- **Schema drift**
  - Code expecting a database column that no longer exists
- **Browser quirks**
  - Safari rendering CSS differently from Chrome
- **Network and timing issues**
  - Race conditions, retries, timeouts
- **That is what the next two layers exist for**

> **Key Takeaway**
> *Unit tests are necessary but never sufficient. The pyramid has three layers for a reason.*

Integration testing — definition

- **Tests that run several units with one real dependency**
  - Usually the database, the framework, or an in-memory message queue
- **Slower than unit tests**
  - Tens to hundreds of milliseconds per test
  - Still fast enough to run on every commit
- **Reveal a different class of bug**
  - Bugs that only appear when components meet
- **Tools**
  - Python — pytest with TestClient (FastAPI) or Django test runner
  - Node.js — supertest with Jest or Vitest

Integration testing — what it catches

- **Wiring mistakes**
  - A route calls the wrong handler function
- **Schema drift**
  - A column was renamed in migration but not in code
- **Validation gaps**
  - A route accepts data its model is supposed to reject
- **Status-code regressions**
  - A 200 OK where a 400 Bad Request should be returned
- **Authorisation regressions**
  - An endpoint forgets to check the user's role

> **Key Takeaway**
> *Skip integration tests and you will ship the same regression three Mondays in a row.*

Setup–teardown discipline

- **The rule**
  - Each test starts from a known state
  - Each test cleans up after itself
- **How to enforce it**
  - Use the framework's setUp / fixture mechanism
  - Spin up a fresh in-memory database per test, or a transaction-rollback wrapper
- **Why this matters**
  - Tests that depend on each other's state are the classic source of flakiness
  - A flaky suite is worse than no suite — engineers stop trusting it

End-to-End testing — definition and cost

- **Definition**
  - Drives a real browser or a real device
  - Exercises the full stack — frontend, backend, database, the lot
- **Cost profile**
  - Slow — seconds per test instead of milliseconds
  - Flaky — networks, animations, timing all conspire
  - Expensive to maintain — every UI redesign breaks selectors
- **Therefore**
  - Keep the count small — roughly ten critical journeys
  - Run on every deploy, not on every commit

End-to-End — selectors and tools

- **Selectors — use data-testid attributes**
  - Owned by the test team, stable across redesigns
  - Never select by CSS class — class names belong to the design team
- **Wait explicitly, never sleep**
  - Wait for an element to be visible, not for a fixed number of seconds
  - Explicit waits are the difference between stable and flaky
- **Tools**
  - JavaScript — Cypress, Playwright (most popular today)
  - Python — Playwright, Selenium

> **Key Takeaway**
> *E2E tests prove the wall stands. Keep them few, keep them stable, run them on every deploy.*

Mobile testing — why harder than web

- **Device matrix**
  - Web — a handful of browsers
  - Mobile — hundreds of Android models, several iOS models
- **Network conditions**
  - WiFi, 5G, 4G, 3G, no signal, captive portals
- **Input methods**
  - Touch, multi-touch, gestures, biometrics, accelerometer
- **Lifecycle interruptions**
  - Phone calls, alarms, low battery, OS updates, app backgrounding

Mobile testing — pyramid tools

- **Same shape, different tools**
- **Unit**
  - JUnit (Java/Kotlin)
  - XCTest (Swift)
- **Integration**
  - Espresso (Android, in-process UI tests)
  - XCUITest (iOS, in-process UI tests)
- **End-to-End**
  - Appium — cross-platform, the de-facto standard
  - Detox — popular in the React Native world

Mobile testing — what it must cover

- **Offline mode**
  - Does the app survive when network drops?
  - Is cached data still readable, is queued data sent on reconnect?
- **Permission denial**
  - What happens when the user says no to camera, location, contacts?
  - App must degrade gracefully, never crash
- **App backgrounding**
  - Does state survive being swiped away and re-opened?
- **Old OS versions**
  - Does the app still work on Android 11 and iOS 15?

> **Key Takeaway**
> *Mobile testing is web testing with the world turned up to eleven — more devices, more states, more interruptions.*

CI / CD — what

- **Continuous Integration (CI)**
  - Every push runs the test suite automatically
  - Catches regressions before they reach the main branch
- **Continuous Delivery (CD)**
  - Every green build is automatically deployable
  - Reduces release stress from a quarterly event to a daily habit
- **Why it matters**
  - The pipeline is the enforcement mechanism that keeps test discipline real
  - Without CI, tests rot

CI / CD — what runs where

- **Pre-commit hook**
  - Linters, formatters, fastest unit tests — on every save
- **CI on push**
  - Full unit and integration suite — on every commit
- **CI on pull request**
  - Above plus a few smoke E2E tests — on every PR
- **Nightly**
  - Full E2E suite plus mobile device farm
- **On deploy**
  - Smoke tests against production after each release

> **Key Takeaway**
> *A test that does not run automatically might as well not exist.*

Lesson 2 recap — the five rules

- **Build a pyramid**
  - Many unit tests, some integration, few E2E
- **Keep tests FIRST**
  - Fast, Isolated, Repeatable, Self-validating, Timely
- **Use AAA**
  - Arrange, Act, Assert — one assertion concept per test
- **Select by data-testid**
  - Stability over beauty — never by CSS class
- **Run them in CI**
  - A test you run by hand is a test you forget to run

> **Key Takeaway**
> *Five rules for tests, five rules for security — and the same mindset underneath both.*

Security and Testing — same coin

- **A security review asks**
  - How could an attacker break this feature?
- **A test asks**
  - How could a future change break this feature?
- **Both are exercises in adversarial imagination**
  - Imagining how the system could fail before it does
  - Writing the failure case down so it stays caught
- **The same mindset, applied at two different moments**

The professional habit

- **Test-Driven Development for security**
- **Step 1 — write the test that proves the feature works**
  - Happy path: valid input, valid output
- **Step 2 — write the test that proves it fails safely**
  - Bad input is rejected
  - Hostile input is rejected
  - Missing permission returns 401 or 403, not 200
- **Step 3 — only then write the production code**

> **Key Takeaway**
> *Engineers who internalise 'how could this go wrong?' on every line become senior engineers; those who skip it spend their careers cleaning up.*

This week's homework

- **Read the OWASP Top 10 (2021) end to end**
  - Pick the three categories that surprised you most
  - Write one paragraph on each in your own words
- **Add a tests/ folder to your current group project**
  - Write five unit tests for a function you have already written
  - Run them. See them pass. Break the function deliberately. See them fail.
- **Try the bcrypt example from the lecture**
  - Notice how slow it is — and remember that slowness is the point

End of deck

Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the IT's Future, Today and Together!

Companion materials: full lecture markdown · twelve runnable code examples · four reference tables (CSV)

Verify external links through VirusTotal: www.virustotal.com

© Cyber.SoHo. For educational use. Use techniques only on systems you own or have permission to test.